

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP VÀ PTNT
TRƯỜNG ĐẠI HỌC THỦY LỢI
PHÂN HIỆU



NGUYỄN XUÂN ĐOÀN KHUÊ
VĂN PHƯỚC NHẬT PHI
ĐẶNG ANH TƯỜNG

PHÂN LOẠI NGƯỜI MẮC BỆNH TÂM THẦN

BÁO CÁO HỌC PHẦN: HỌC MÁY

HỒ CHÍ MINH, NĂM 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP VÀ PTNT
TRƯỜNG ĐẠI HỌC THỦY LỢI
PHÂN HIỆU

PHÂN LOẠI NGƯỜI MẮC BỆNH TÂM THẦN

Họ và tên:	<i>NGUYỄN XUÂN ĐOÀN KHUÊ</i>	<i>VĂN PHƯỚC NHẬT PHI</i>	<i>ĐẶNG ANH TƯỜNG</i>
Mã sinh viên:	<i>2351067097</i>	<i>2351067107</i>	<i>2351067120</i>
Lớp:	<i>S26-K65CNTT</i>	<i>S26-K65CNTT</i>	<i>S26-K65CNTT</i>
Ngành:	Công nghệ thông tin	Công nghệ thông tin	Công nghệ thông tin

Giảng viên hướng dẫn: ThS Vũ Thị Hạnh

HỒ CHÍ MINH, NĂM 2025

BẢNG PHÂN CÔNG THÀNH VIÊN

Họ và tên:	<i>NGUYỄN XUÂN ĐOÀN KHUÊ</i>	<i>VĂN PHƯỚC NHẬT PHI</i>	<i>ĐẶNG ANH TUỜNG</i>
Mô tả dữ liệu sử dụng + Tiền xử lý	Hỗ Trợ	Thực hiện	Hỗ Trợ
Decision Tree	Thực Hiện	Hỗ Trợ	Hỗ Trợ
Random Forest	Hỗ Trợ	Hỗ Trợ	Thực Hiện
Web/App	Hỗ Trợ	Hỗ Trợ	Thực Hiện
LightGBM	Hỗ Trợ	Thực Hiện	Hỗ Trợ
XGBoost	Hỗ Trợ	Thực Hiện	Hỗ Trợ
Đánh giá	Thực Hiện	Thực Hiện	Thực Hiện

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin trân trọng cảm ơn giảng viên Vũ Thị Hạnh - người đã trực tiếp chỉ bảo, hướng dẫn em trong quá trình hoàn thành bài tiểu luận này.

Chúng em cũng xin được gửi lời cảm ơn đến quý thầy giáo trường Phân hiệu Đại học Thủy Lợi, đặc biệt là các thầy khoa Công nghệ thông tin - những người đã truyền lửa và giảng dạy kiến thức cho em suốt thời gian qua.

Mặc dù đã có những đầu tư nhất định trong quá trình làm bài song cũng khó có thể tránh khỏi những sai sót, em kính mong nhận được ý kiến đóng góp của quý thầy cô để bài tiểu luận được hoàn thiện hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

Giới thiệu bài toán cần làm	7
Đối tượng nghiên cứu.....	8
Phạm vi bài toán.....	8
Tóm tắt kết quả đạt được.....	8
Mục tiêu của đề tài	9
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT HỌC MÁY.....	9
1.1. Tổng quan về học máy	9
1.2. Học có giám sát.....	9
1.3. Các thuật toán phân loại được sử dụng	10
CHƯƠNG 2 PHÂN TÍCH & XÂY DỰNG MÔ HÌNH.....	10
2.1. Mô tả tập dữ liệu	10
2.2. Mô tả các thuộc tính trong dữ liệu	10
2.3. Trực quan hóa và phân tích dữ liệu ban đầu	11
2.4. Xử lý dữ liệu thiếu (Missing Values).....	12
2.5. Loại bỏ thuộc tính dư thừa	13
2.6. Mã hóa dữ liệu	13
2.6.1. Mã hóa thuộc tính nhị phân	14
2.6.2. Mã hóa thuộc tính thứ bậc	14
2.6.3. Mã hóa thuộc tính danh mục.....	14
2.6.4. Kết Quả Xử Lý.....	15
2.7. Chuẩn bị dữ liệu cho mô hình.....	15
2.8 Kết luận	15
CHƯƠNG 3:	16
PHƯƠNG PHÁP VÀ MÔ HÌNH HỌC MÁY ÁP DỤNG	16
3.1. Bài toán học máy đặt ra.....	16
3.2. Quy trình xây dựng mô hình	16
3.3.1. Decision Tree (Cây quyết định)	16
3.3.2. Random Forest (Rừng ngẫu nhiên).....	17
.....	17
3.3.4. LightGBM (Light Gradient Boosting Machine)	17
3.3.5. XGBoost (Extreme Gradient Boosting)	17
3.4. Chỉ số đánh giá mô hình.....	18
3.5. Lý do lựa chọn các mô hình	18

3.6. Tối ưu tham số mô hình	18
3.7. Kết luận chương.....	18
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ NHẬN XÉT.....	19
4.1. Thiết lập thực nghiệm.....	19
4.2. Kết quả mô hình Decision Tree	19
4.2.1. Kết quả trước khi tối ưu tham số.....	19
4.2.2. Kết quả sau khi tối ưu tham số.....	22
4.3. Kết quả mô hình Random Forest.....	24
4.3.1. Trước khi tối ưu tham số.....	24
4.3.2. Sau khi tối ưu tham số	26
4.5. Kết quả mô hình LightGBM	28
4.5.1. Trước tuning	28
4.5.2. Sau tuning.....	30
4.6. Kết quả mô hình XGBoost.....	32
4.6.1. Trước tuning	32
4.6.2. Sau tuning	34
4.7. So sánh và Nhận xét tổng thể các mô hình.....	36
Naive Bayes có hiệu suất thấp nhất.....	Error! Bookmark not defined.
Decision Tree và Random Forest cho kết quả khá tốt	37
LightGBM và XGBoost cho hiệu suất cao nhất	38
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	38
5.1. Kết luận chung về đề tài	38
5.2. Những hạn chế còn tồn tại	39
5.3. Hướng phát triển trong tương lai.....	39
TÀI LIỆU THAM KHẢO	39

Mở Đầu

Giới thiệu bài toán cần làm

Trong những năm gần đây, sức khỏe tâm thần đang trở thành một trong những vấn đề được xã hội đặc biệt quan tâm, nhất là trong bối cảnh áp lực học tập, công việc và các mối quan hệ xã hội ngày càng gia tăng. Các rối loạn về tâm lý như căng thẳng, lo âu, trầm cảm nếu không được phát hiện và hỗ trợ kịp thời có thể ảnh hưởng nghiêm trọng đến chất lượng cuộc sống, hiệu suất học tập và lao động của con người.

Với sự phát triển mạnh mẽ của Học máy (Machine Learning – mô hình cho phép máy học từ dữ liệu), việc phân tích dữ liệu sức khỏe tâm thần để tìm ra các yếu tố ảnh hưởng và xây dựng mô hình dự đoán đã trở nên khả thi và hiệu quả hơn. Do đó, đề tài phân tích dữ liệu sức khỏe tâm thần được lựa chọn nhằm ứng dụng các thuật toán học máy để hỗ trợ đánh giá và dự đoán tình trạng tâm lý của con người dựa trên dữ liệu khảo sát thu thập được.

Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là những cá nhân tham gia khảo sát sức khỏe tâm thần, với các thông tin liên quan đến đặc điểm cá nhân, lối sống, thói quen sinh hoạt và một số yếu tố phản ánh tình trạng tâm lý. Dữ liệu này phản ánh mối liên hệ giữa các yếu tố đời sống hằng ngày và nguy cơ gặp các vấn đề về sức khỏe tâm thần, từ đó làm cơ sở cho việc phân tích và xây dựng mô hình dự đoán.

Phạm vi bài toán

Phạm vi của đề tài tập trung vào việc phân tích và xử lý tập dữ liệu Mental Health Dataset được thu thập từ nền tảng Kaggle. Đề tài thực hiện các bước tiền xử lý dữ liệu, trực quan hóa dữ liệu và áp dụng một số thuật toán học máy phổ biến để xây dựng mô hình dự đoán tình trạng sức khỏe tâm thần.

Đề tài không đi sâu vào chẩn đoán y khoa hay điều trị lâm sàng, mà chỉ dừng lại ở mức độ phân tích dữ liệu và hỗ trợ dự đoán nhằm cung cấp thông tin tham khảo cho việc nhận diện sớm các vấn đề tâm lý.

Tóm tắt kết quả đạt được

Sau quá trình nghiên cứu và thực hiện đề tài, nhóm đã đạt được một số kết quả chính như sau:

- Phân tích được các đặc điểm và phân bố của dữ liệu liên quan đến sức khỏe tâm thần.
- Xác định được mối quan hệ giữa một số yếu tố cá nhân, lối sống và tình trạng tâm lý của người tham gia khảo sát.
- Xây dựng và huấn luyện các mô hình học máy nhằm dự đoán nguy cơ gặp vấn đề về sức khỏe tâm thần.
- Đánh giá và so sánh hiệu quả của các mô hình dựa trên các chỉ số như độ chính xác và khả năng phân loại.

Mục tiêu của đề tài

Mục tiêu chính của đề tài là ứng dụng các kỹ thuật học máy để phân tích dữ liệu sức khỏe tâm thần, từ đó xây dựng mô hình dự đoán tình trạng tâm lý của con người.. Các mục tiêu cụ thể bao gồm:

- Khám phá và phân tích các yếu tố ảnh hưởng đến sức khỏe tâm thần.
- Xây dựng mô hình học máy nhằm dự đoán nguy cơ gặp vấn đề tâm lý dựa trên dữ liệu khảo sát.
- Đánh giá hiệu quả của các thuật toán học máy được sử dụng trong bài toán.
- Góp phần cung cấp cơ sở tham khảo cho việc nghiên cứu và hỗ trợ nhận diện sớm các vấn đề về sức khỏe tâm thần trong thực tế.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT HỌC MÁY

1.1. Tổng quan về học máy

Học máy (*Machine Learning – phương pháp cho phép máy tính học từ dữ liệu*) là một nhánh của trí tuệ nhân tạo, tập trung vào việc xây dựng các mô hình có khả năng học từ dữ liệu và đưa ra dự đoán hoặc quyết định mà không cần lập trình chi tiết cho từng trường hợp cụ thể. Học máy được ứng dụng rộng rãi trong nhiều lĩnh vực như y tế, tài chính, giáo dục và khoa học xã hội.

Trong các bài toán phân loại, học máy giúp tự động phân nhóm các đối tượng dữ liệu dựa trên những đặc trưng đầu vào, từ đó hỗ trợ quá trình ra quyết định một cách hiệu quả và chính xác hơn.

1.2. Học có giám sát

Học có giám sát (*Supervised Learning – phương pháp học với dữ liệu đã được gán nhãn*) là loại học máy trong đó mỗi mẫu dữ liệu đầu vào đều đi kèm với một nhãn đầu ra xác định. Mục tiêu của mô hình là học được mối quan hệ giữa dữ liệu đầu vào và nhãn để có thể dự đoán nhãn cho các dữ liệu chưa từng xuất hiện trước đó.

Trong đồ án này, học có giám sát được sử dụng để giải quyết bài toán phân loại tình trạng sức khỏe tâm thần dựa trên các đặc trưng thu thập từ dữ liệu khảo sát.

1.3. Các thuật toán phân loại được sử dụng

Trong đồ án này, nhóm sử dụng các thuật toán học máy có giám sát để giải quyết bài toán phân loại, bao gồm:

- **Decision Tree** (*Cây quyết định – mô hình phân loại dựa trên cấu trúc cây*): thuật toán xây dựng mô hình dưới dạng cây, trong đó mỗi nút biểu diễn một điều kiện phân chia dữ liệu. Mô hình có ưu điểm là dễ hiểu và dễ diễn giải.
- **Random Forest** (*Rừng ngẫu nhiên – mô hình tập hợp nhiều cây quyết định*): kết hợp nhiều cây quyết định được huấn luyện độc lập nhằm nâng cao độ chính xác và giảm hiện tượng quá khớp so với cây quyết định đơn lẻ.
- **LightGBM** (Light Gradient Boosting Machine – mô hình tăng cường gradient): là thuật toán học máy dựa trên phương pháp gradient boosting, kết hợp nhiều mô hình yếu theo hướng tối ưu hóa hàm mất mát, được thiết kế để huấn luyện nhanh và sử dụng tài nguyên hiệu quả trong các bài toán phân loại.
- **XGBoost** (Extreme Gradient Boosting – mô hình tăng cường gradient): là thuật toán học máy mạnh mẽ, kết hợp nhiều mô hình yếu theo hướng tối ưu hóa hàm mất mát, cho hiệu suất cao trong các bài toán phân loại.

Các thuật toán trên được sử dụng để huấn luyện và so sánh hiệu quả phân loại trong bài toán dự đoán tình trạng sức khỏe tâm thần.

CHƯƠNG 2 PHÂN TÍCH & XÂY DỰNG MÔ HÌNH

2.1. Mô tả tập dữ liệu

Trong đồ án này, nhóm sử dụng Mental Health Dataset được thu thập từ nền tảng Kaggle. Tập dữ liệu bao gồm thông tin khảo sát của nhiều cá nhân liên quan đến sức khỏe tâm thần, phản ánh các yếu tố cá nhân, lối sống, thói quen sinh hoạt và tình trạng tâm lý.

Mục tiêu của tập dữ liệu là hỗ trợ phân tích mối quan hệ giữa các đặc trưng đầu vào và nhu cầu điều trị sức khỏe tâm thần của người tham gia khảo sát. Biến mục tiêu trong bài toán là Treatment, thể hiện việc một cá nhân có hay không có nhu cầu điều trị liên quan đến sức khỏe tâm thần.

2.2. Mô tả các thuộc tính trong dữ liệu

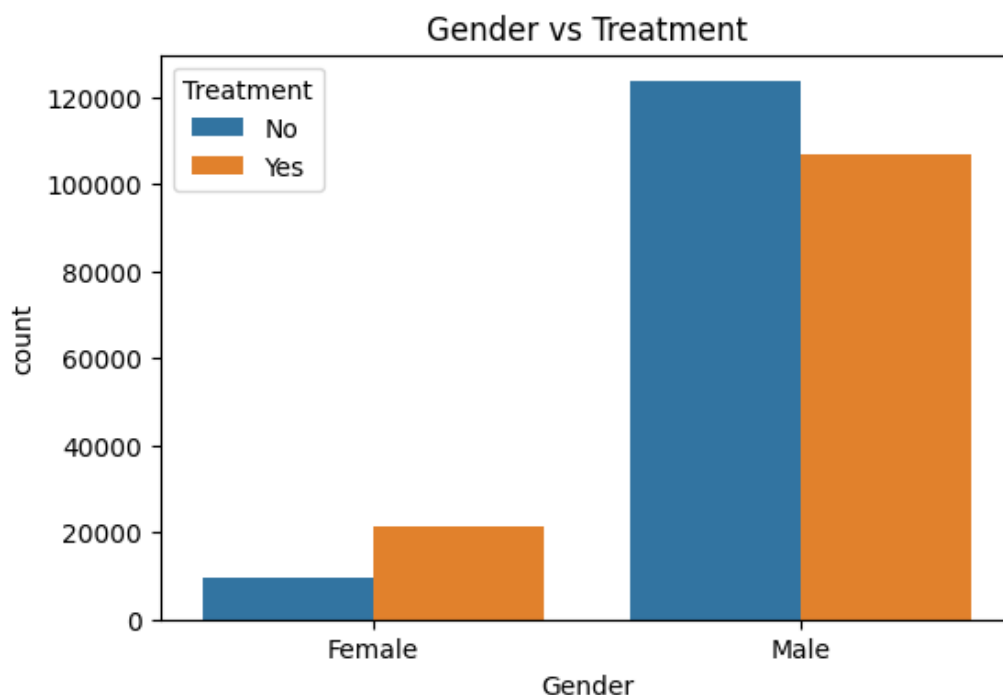
Tập dữ liệu bao gồm các nhóm thuộc tính chính sau:

- Thông tin cá nhân: giới tính (Gender), quốc gia (Country), nghề nghiệp (Occupation), tình trạng tự làm việc (SelfEmployed).
- Tiền sử và yếu tố tâm lý: tiền sử bệnh tâm thần (MentalHealthHistory), tiền sử gia đình (FamilyHistory), thay đổi thói quen (HabitsChange), mức độ căng thẳng (IncreasingStress), thay đổi tâm trạng (MoodSwings).
- Yếu tố xã hội và hành vi: số ngày ở trong nhà (DaysIndoors), mức độ suy giảm hứng thú công việc (WorkInterest), mức độ yếu kém trong giao tiếp xã hội (SocialWeakness), khả năng đối phó khó khăn (CopingStruggles).
- Tiếp cận chăm sóc & phỏng vấn: Mental HealthInterview, CareOptions
- Biến mục tiêu: Treatment – thể hiện cá nhân có hoặc không có nhu cầu điều trị sức khỏe tâm thần.

Các thuộc tính trong tập dữ liệu tồn tại ở nhiều dạng khác nhau như nhị phân, thứ bậc (ordinal) và danh mục (nominal), do đó cần thực hiện các bước tiền xử lý phù hợp trước khi đưa vào mô hình học máy.

2.3. Trực quan hóa và phân tích dữ liệu ban đầu

Nhóm tiến hành trực quan hóa dữ liệu nhằm quan sát phân bố và mối quan hệ giữa các đặc trưng với biến mục tiêu.



Từ Hình trên có thể thấy sự phân bố giới tính trong tập dữ liệu không đồng đều, trong đó nam giới chiếm tỷ lệ lớn hơn nữ giới. Đồng thời, ở cả hai giới đều tồn tại tỷ lệ đáng kể các cá nhân có nhu cầu điều trị (Treatment = 1), cho thấy giới tính có thể là một yếu tố liên quan đến sức khỏe tâm thần và cần được xem xét trong quá trình xây dựng mô hình.

2.4. Xử lý dữ liệu thiếu (Missing Values)

Trước khi huấn luyện mô hình, nhóm tiến hành kiểm tra dữ liệu thiếu trong tập dữ liệu. Kết quả cho thấy một số thuộc tính tồn tại giá trị bị thiếu, trong đó đáng chú ý là cột **SelfEmployed**.

```
#1. XỬ LÝ MISSING VALUES
#Kiểm tra số lượng missing
print("\nSố lượng missing theo cột:")
print(df.isnull().sum())
```

```
Số lượng missing theo cột:
Gender                0
Country              0
Occupation            0
SelfEmployed         4356
FamilyHistory        0
Treatment            0
DaysIndoors          0
HabitsChange         0
MentalHealthHistory  0
IncreasingStress     0
MoodSwings           0
SocialWeakness       0
CopingStruggles      0
WorkInterest         0
SocialWeakness.1     0
MentalHealthInterview 0
CareOptions          0
dtype: int64
```

Để đảm bảo tính toàn vẹn của dữ liệu và tránh làm giảm số lượng mẫu, nhóm lựa chọn phương pháp **điền giá trị phổ biến nhất (mode)** cho cột **SelfEmployed**. Phương pháp này phù hợp với dữ liệu dạng phân loại và giúp giữ nguyên phân bố tổng thể của dữ liệu.

Sau khi thực hiện, tập dữ liệu không còn giá trị thiếu và sẵn sàng cho các bước xử lý tiếp theo.

```
#Điền giá trị thiếu cho cột SelfEmployed
#Điền bằng giá trị phổ biến nhất (mode)
df['SelfEmployed'] = df['SelfEmployed'].fillna(df['SelfEmployed'].mode()[0])

#Kiểm tra sau khi điền dữ liệu
print("\nSố lượng missing sau khi impute:")
print(df.isna().sum())
```

```
Số lượng missing sau khi impute:
Gender          0
Country         0
Occupation      0
SelfEmployed    0
FamilyHistory   0
Treatment       0
DaysIndoors     0
HabitsChange    0
MentalHealthHistory 0
IncreasingStress 0
MoodSwings      0
SocialWeakness  0
CopingStruggles 0
WorkInterest    0
SocialWeakness.1 0
MentalHealthInterview 0
CareOptions     0
dtype: int64
```

2.5. Loại bỏ thuộc tính dư thừa

Trong quá trình kiểm tra dữ liệu, nhóm phát hiện hai cột **SocialWeakness** và **SocialWeakness.1** có nội dung trùng lặp hoàn toàn. Việc giữ lại cả hai cột có thể gây dư thừa thông tin và ảnh hưởng đến hiệu quả của mô hình.

```
#2. LOẠI BỎ CỘT
#Có hai cột giống tên SocialWeakness và SocialWeakness.1
#Kiểm tra xem dữ liệu có bị trùng lặp không
(df['SocialWeakness'] == df['SocialWeakness.1']).value_counts()
```

```
True    261328
Name: count, dtype: int64
```

Kết quả trả về là True => dữ liệu bị trùng

Do đó, nhóm quyết định **loại bỏ cột SocialWeakness.1**, chỉ giữ lại một cột đại diện nhằm đảm bảo dữ liệu gọn nhẹ và tránh nhiễu.

```
#Chọn xóa cột SocialWeakness.1 do dữ liệu trùng lặp
df = df.drop(columns=['SocialWeakness.1'])

#Kiểm tra cột SocialWeakness.1 còn tồn tại không
print('SocialWeakness.1' in df.columns) # Trả về True/False
```

False

2.6. Mã hóa dữ liệu

Do các thuật toán học máy yêu cầu dữ liệu đầu vào ở dạng số, nhóm tiến hành mã hóa dữ liệu theo từng loại thuộc tính:

2.6.1. Mã hóa thuộc tính nhị phân

Các thuộc tính nhị phân như **SelfEmployed**, **FamilyHistory**, **Treatment**, **CopingStruggles** được ánh xạ từ giá trị *Yes/No* sang *1/0*. Thuộc tính **Gender** cũng được mã hóa dưới dạng số.

```
#3. MÃ HÓA DỮ LIỆU
#3.1 Xử lý các cột Binary bằng Mapping (yes/no -> 1/0)
for col in ['SelfEmployed', 'FamilyHistory', 'Treatment', 'CopingStruggles']:
    df[col] = df[col].map({'No': 0, 'Yes': 1})

df['Gender'] = df['Gender'].map({'Male': 0, 'Female': 1})
```

2.6.2. Mã hóa thuộc tính thứ bậc

Một số thuộc tính có tính thứ bậc được mã hóa theo mức độ tăng dần, ví dụ:

- **DaysIndoors**: từ “Go out Every day” đến “More than 2 months”.
- **MoodSwings**: từ mức thấp đến mức cao.

Việc mã hóa theo thứ bậc giúp mô hình học được mối quan hệ tăng giảm giữa các mức giá trị.

```
#3.2 Xử lý các cột Ordinal bằng Ordinal encoding
days_map = {'Go out Every day': 0, '1-14 days': 1, '15-30 days': 2, '31-60 days': 3, 'More than 2 months': 4}

level_map = {'Low': 0, 'Medium': 1, 'High': 2}

df['DaysIndoors'] = df['DaysIndoors'].map(days_map)
df['MoodSwings'] = df['MoodSwings'].map(level_map)

#Kiểm tra sau xử lý có còn dữ liệu NaN do lỗi không
print(df[['SelfEmployed', 'FamilyHistory', 'Treatment', 'CopingStruggles', 'Gender', 'DaysIndoors', 'MoodSwings']].isna().sum())
```

2.6.3. Mã hóa thuộc tính danh mục

Các thuộc tính danh mục nhiều giá trị như **Country**, **Occupation** và một số yếu tố tâm lý khác được xử lý bằng phương pháp **One-Hot Encoding**. Phương pháp này giúp tránh tạo ra mối quan hệ thứ bậc giả giữa các giá trị danh mục.

```
#3.3 Xử lý các cột còn lại bằng one-hot encoding
multi_cols = ['MentalHealthHistory', 'HabitsChange', 'IncreasingStress',
              'SocialWeakness', 'WorkInterest', 'MentalHealthInterview', 'CareOptions']

nominal_cols = ['Country', 'Occupation']

df = pd.get_dummies(df, columns=multi_cols + nominal_cols, drop_first=True)
```

2.6.4. Kết Quả Xử Lý

	gender	seitemployed	familyhistory	treatment	daysindoors	moodswings	copingsstruggles	mentalhealthhistory_no	mentalhealth
0	1	0	1	0	2	2	1	False	
1	1	0	1	0	2	2	1	False	
2	1	0	0	1	4	1	0	True	
3	1	0	1	1	0	1	0	False	
4	1	0	0	1	4	1	0	True	
5	1	0	0	0	0	2	0	False	
6	1	0	1	0	1	2	1	False	
7	1	0	1	1	2	1	1	False	
8	1	0	1	1	0	1	1	False	
9	1	0	1	1	1	1	1	False	

2.7. Chuẩn bị dữ liệu cho mô hình

Sau khi hoàn tất các bước tiền xử lý, tập dữ liệu được chia thành:

- Tập huấn luyện (80%)
- Tập kiểm tra (20%)

Việc chia dữ liệu được thực hiện theo phương pháp ngẫu nhiên có phân tầng theo biến mục tiêu nhằm đảm bảo tỷ lệ các lớp được giữ ổn định giữa hai tập dữ liệu.

Tập dữ liệu sau tiền xử lý đảm bảo:

- Không còn giá trị thiếu
- Các thuộc tính đều ở dạng số
- Phù hợp để đưa vào huấn luyện các mô hình học máy có giám sát

```
#Chuẩn bị dữ liệu
X = df.drop('Treatment', axis=1)
y = df['Treatment']
#Train 80, Test 20
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

2.8 Kết luận

Sau các bước tiền xử lý, dữ liệu đã được làm sạch, chuyển đổi sang dạng số và sẵn sàng để đưa vào huấn luyện các mô hình học máy có giám sát nhằm giải quyết bài toán phân loại tình trạng sức khỏe tâm thần

CHƯƠNG 3:

PHƯƠNG PHÁP VÀ MÔ HÌNH HỌC MÁY ÁP DỤNG

3.1. Bài toán học máy đặt ra

Dựa trên tập dữ liệu sức khỏe tâm thần đã được tiền xử lý ở Chương 2, bài toán được xác định là **bài toán phân loại nhị phân có giám sát**.

Mục tiêu của bài toán là xây dựng mô hình học máy nhằm **dự đoán biến mục tiêu Treatment**, trong đó:

- **Treatment = 1**: Cá nhân có nhu cầu điều trị liên quan đến sức khỏe tâm thần
- **Treatment = 0**: Cá nhân không có nhu cầu điều trị

Các đặc trưng đầu vào bao gồm thông tin cá nhân, thói quen sinh hoạt, yếu tố tâm lý và xã hội của người tham gia khảo sát.

3.2. Quy trình xây dựng mô hình

Quy trình xây dựng mô hình học máy trong đồ án được thực hiện theo các bước sau:

1. Chuẩn bị tập dữ liệu đã được tiền xử lý hoàn chỉnh
2. Chia dữ liệu thành tập huấn luyện và tập kiểm tra theo tỷ lệ 80% – 20%
3. Huấn luyện các mô hình học máy có giám sát
4. Đánh giá hiệu quả mô hình trên tập kiểm tra
5. Tối ưu tham số (tuning) nhằm cải thiện hiệu suất
6. So sánh và phân tích kết quả giữa các mô hình

3.3.1. Decision Tree (Cây quyết định)

Decision Tree là mô hình phân loại dựa trên cấu trúc cây, trong đó mỗi nút biểu diễn một điều kiện phân chia dữ liệu theo một thuộc tính nhất định. Quá trình phân nhánh tiếp tục cho đến khi đạt được các nút lá đại diện cho kết quả phân loại.

Ưu điểm:

- Dễ hiểu, dễ diễn giải
- Không yêu cầu chuẩn hóa dữ liệu
- Phù hợp cho việc phân tích dữ liệu ban đầu

Nhược điểm:

- Dễ bị quá khớp (overfitting) nếu cây quá sâu

Trong đồ án, mô hình Decision Tree được huấn luyện ở hai phiên bản:

- Mô hình cơ bản (baseline)
- Mô hình tối ưu tham số bằng RandomizedSearchCV

3.3.2. Random Forest (Rừng ngẫu nhiên)

Random Forest là mô hình tập hợp (ensemble) bao gồm nhiều cây quyết định được huấn luyện độc lập trên các tập con khác nhau của dữ liệu và đặc trưng. Kết quả dự đoán cuối cùng được đưa ra dựa trên nguyên tắc bỏ phiếu đa số.

Ưu điểm:

- Giảm hiện tượng quá khớp so với Decision Tree
- Độ chính xác cao và ổn định
- Hoạt động tốt với dữ liệu có nhiều đặc trưng

Trong đồ án, Random Forest được huấn luyện với:

- Mô hình cơ bản
- Mô hình tối ưu tham số nhằm cải thiện độ chính xác

3.3.4. LightGBM (Light Gradient Boosting Machine)

LightGBM là thuật toán học máy dựa trên phương pháp **Gradient Boosting**, trong đó nhiều mô hình yếu được kết hợp tuần tự để tối ưu hóa hàm mất mát. LightGBM được thiết kế để huấn luyện nhanh, sử dụng bộ nhớ hiệu quả và phù hợp với tập dữ liệu lớn.

Ưu điểm:

- Tốc độ huấn luyện nhanh
- Khả năng xử lý dữ liệu nhiều chiều tốt
- Hiệu suất cao trong các bài toán phân loại

Trong đồ án, LightGBM được huấn luyện với cấu hình ban đầu và sau đó được tối ưu tham số bằng RandomizedSearchCV để nâng cao kết quả dự đoán.

3.3.5. XGBoost (Extreme Gradient Boosting)

XGBoost là một trong những thuật toán Gradient Boosting mạnh mẽ và phổ biến nhất hiện nay. Thuật toán này sử dụng cơ chế tăng cường gradient kết hợp với các kỹ thuật regularization nhằm giảm overfitting và nâng cao khả năng tổng quát hóa.

Ưu điểm:

- Độ chính xác cao
- Khả năng kiểm soát quá khớp tốt

- Hoạt động hiệu quả trong các bài toán phân loại phức tạp

Trong đồ án, XGBoost được sử dụng làm mô hình chính để so sánh, bao gồm:

- Mô hình XGBoost cơ bản
- Mô hình XGBoost tối ưu tham số

3.4. Chỉ số đánh giá mô hình

Để đánh giá hiệu quả của các mô hình học máy, nhóm sử dụng các chỉ số sau:

- **Accuracy (Độ chính xác):** tỷ lệ dự đoán đúng trên tổng số mẫu
- **ROC AUC:** đo lường khả năng phân biệt giữa hai lớp
- **Confusion Matrix:** đánh giá chi tiết số lượng dự đoán đúng và sai cho từng lớp
- **Classification Report:** bao gồm Precision, Recall và F1-score

Các chỉ số này giúp so sánh toàn diện hiệu suất giữa các mô hình khác nhau.

3.5. Lý do lựa chọn các mô hình

Nhóm lựa chọn các mô hình từ đơn giản đến phức tạp nhằm đánh giá ảnh hưởng của độ phức tạp mô hình đến khả năng dự đoán. Decision Tree và Naive Bayes được sử dụng làm mô hình cơ sở, trong khi Random Forest, LightGBM và XGBoost đại diện cho nhóm mô hình ensemble có khả năng nâng cao hiệu suất phân loại. Việc so sánh này giúp đánh giá toàn diện hiệu quả của các phương pháp học máy trong bài toán sức khỏe tâm thần.

3.6. Tối ưu tham số mô hình

Để cải thiện hiệu suất dự đoán, nhóm sử dụng phương pháp tìm kiếm tham số ngẫu nhiên (RandomizedSearchCV) kết hợp với kiểm tra chéo nhằm xác định bộ tham số tối ưu cho các mô hình. Việc tuning giúp cân bằng giữa độ chính xác và khả năng tổng quát hóa, đồng thời giảm nguy cơ quá khớp.

3.7. Kết luận chương

Trong chương này, nhóm đã trình bày bài toán học máy, quy trình xây dựng mô hình và mô tả chi tiết các thuật toán được sử dụng trong đồ án. Các mô hình này sẽ được đánh giá và so sánh hiệu quả trong **Chương 4 – Kết quả thực nghiệm và nhận xét**.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ NHẬN XÉT

4.1. Thiết lập thực nghiệm

Trong đồ án này, các mô hình học máy được triển khai và đánh giá trong môi trường Python. Sau khi hoàn tất quá trình tiền xử lý dữ liệu, tập dữ liệu được chia thành hai phần:

- **Tập huấn luyện (80%)**
- **Tập kiểm tra (20%)**

Việc chia dữ liệu được thực hiện theo phương pháp phân tầng nhằm đảm bảo tỷ lệ phân bố của biến mục tiêu Treatment được giữ ổn định giữa hai tập dữ liệu.

Các mô hình được đánh giá thông qua các chỉ số phổ biến trong bài toán phân loại nhị phân, bao gồm:

- **Accuracy**
- **ROC AUC**
- **Confusion Matrix**
- **Precision, Recall, F1-score**

Mục tiêu của quá trình thực nghiệm là đánh giá khả năng dự đoán và so sánh hiệu quả giữa các mô hình học máy khác nhau khi áp dụng vào bài toán sức khỏe tâm thần.

4.2. Kết quả mô hình Decision Tree

4.2.1. Kết quả trước khi tối ưu tham số

Mô hình Decision Tree cơ bản được huấn luyện với các tham số mặc định. Kết quả cho thấy mô hình có khả năng học nhanh nhưng dễ bị quá khớp do cấu trúc cây phức tạp, dẫn đến hiệu suất dự đoán chưa cao trên tập kiểm tra.

```
# Khởi tạo mô hình Decision Tree với độ sâu nhỏ = 3
model = DecisionTreeClassifier(max_depth=3, random_state=42)
# Huấn luyện mô hình
model.fit(X_train, y_train)
```

```

Decision Tree (Baseline)
Accuracy : 0.7081
ROC AUC  : 0.7549
Classification Report:

```

	precision	recall	f1-score	support
0	0.74	0.66	0.70	26592
1	0.68	0.76	0.72	25674
accuracy			0.71	52266
macro avg	0.71	0.71	0.71	52266
weighted avg	0.71	0.71	0.71	52266

Accuracy (0.7081) → ~70.81%, thấp hơn Random Forest

ROC AUC (0.7549) → Khả năng phân biệt 2 lớp khá tốt

Class 0 (không cần treatment)

-Precision: 0.74 → 74% dự đoán đúng.

-Recall: 0.66 → Phát hiện 66% số thực sự lớp 0, trung bình khá.

-F1-score: 0.7 → Trung bình.

Class 1 (cần treatment)

-Precision: 0.68 → 68% dự đoán đúng, trung bình khá.

-Recall: 0.76 → Phát hiện 76% số thực sự lớp 1, mức khá.

-F1-score: 0.72 → Trung bình, cân đối.

Macro avg → 0.71 (precision), 0.71 (recall), 0.71 (F1) → Cân đối giữa 2 lớp.

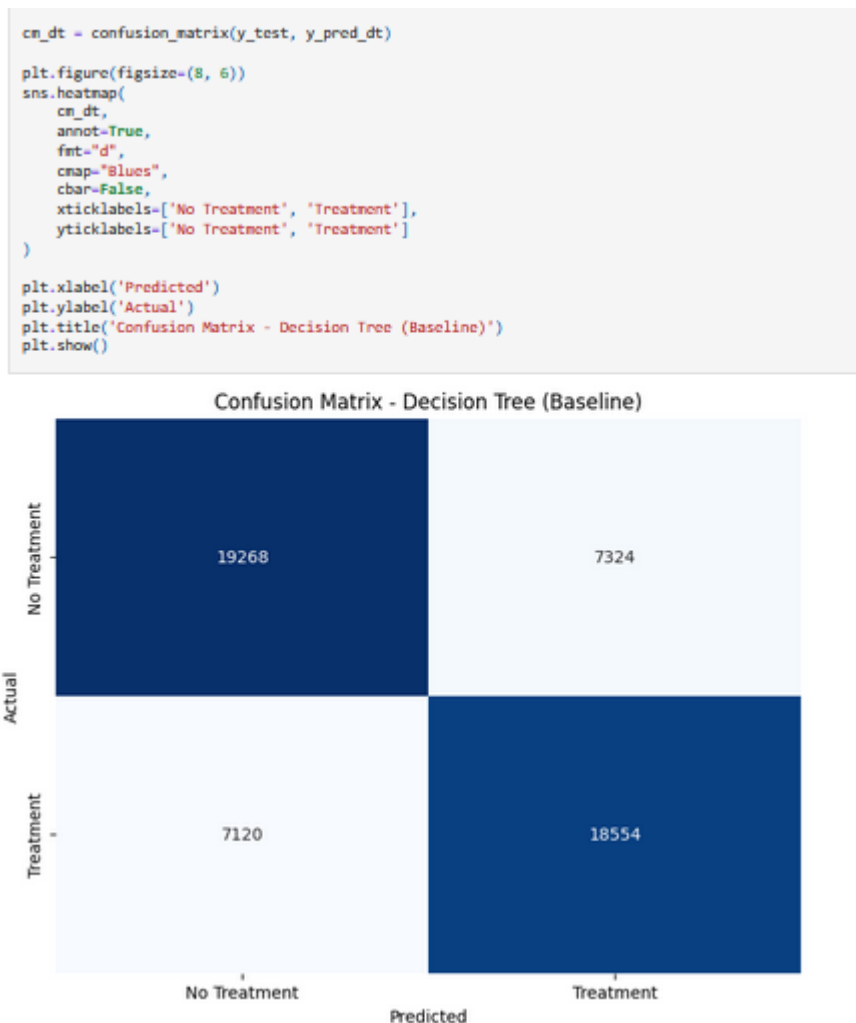
Weighted avg → 0.71, 0.71, 0.71 → Trung bình theo số lượng mẫu.

```

importances = pd.Series(
    model.feature_importances_,          # Giá trị importance từ mô hình
    index=X_train.columns                # Tên các cột features
)
# Các feature có importance > 0 là các feature
# thực sự được Decision Tree sử dụng để ra quyết định
important_features = importances[importances > 0]
print("Số features có tác động:", important_features.shape[0])
print("\n=== Features có ảnh hưởng ===")
print(important_features.sort_values(ascending=False))
# Các feature có importance = 0:
# - Không được sử dụng trong bất kỳ node nào
# - Có thể là feature dư thừa hoặc gây nhiễu
non_important_features = importances[importances == 0]
print("Số features không ảnh hưởng:", non_important_features.shape[0])

```

Xem các Features nào có tác động đến mô hình trước khi tuning , những features có tác động đều > 0 còn lại thì không tác động



Nhận xét: Ma trận nhầm lẫn cho thấy Decision Tree gặp khó khăn trong việc phân biệt chính xác hai lớp, đặc biệt là lớp có nhu cầu điều trị.

4.2.2. Kết quả sau khi tối ưu tham số

Sau khi áp dụng RandomizedSearchCV để tối ưu các tham số như độ sâu cây và số mẫu tối thiểu tại mỗi nút, hiệu suất của mô hình Decision Tree được cải thiện đáng kể.

```

# Khoảng tham số để tìm kiếm ngẫu nhiên (rộng nhưng thực tế)
param_dist = {
    "max_depth": [None, 3, 5, 7, 10, 15, 20, 30, 40, 50, 60, 70],      # Độ sâu cây
    "min_samples_split": [2, 5, 10, 20, 50, 100, 110, 120, 150, 200],  # Số mẫu tối thiểu để split node
    "min_samples_leaf": [1, 2, 5, 10, 20, 50, 100, 110, 120, 150, 200], # Số mẫu tối thiểu ở lá
    "criterion": ["gini", "entropy"]                                  # Đo lường impurity
}

# Khởi tạo RandomizedSearchCV
search = RandomizedSearchCV(
    DecisionTreeClassifier(random_state=42), # Mô hình cơ sở
    param_distributions=param_dist,          # Khoảng tham số
    n_iter=100,                             # Số lần ngẫu nhiên 100 tổ hợp (có thể tăng nếu máy mạnh)
    cv=5,                                   # 5-fold cross-validation
    scoring="roc_auc",                     # Tiêu chí đánh giá: AUC tốt cho bài toán này
    random_state=42,                       # Cố định kết quả random
    n_jobs=-1,                             # Chạy song song trên tất cả lõi CPU
)

```

```

Decision Tree (Tuned)
Accuracy : 0.7722
ROC AUC  : 0.8610
Classification Report:

```

	precision	recall	f1-score	support
0	0.81	0.72	0.76	26592
1	0.74	0.83	0.78	25674
accuracy			0.77	52266
macro avg	0.78	0.77	0.77	52266
weighted avg	0.78	0.77	0.77	52266

Accuracy (0.7722) → ~77.22%, cải thiện đáng kể

ROC AUC (0.841) → Khả năng phân biệt 2 lớp tốt

Class 0 (không cần treatment)

-Precision: 0.81 → 81% dự đoán đúng.

-Recall: 0.72 → Phát hiện 72% số thực sự lớp 0.

-F1-score: 0.76 → Trung bình tốt.

Class 1 (cần treatment)

-Precision: 0.74 → 74% dự đoán đúng.

-Recall: 0.83 → Phát hiện 83% số thực sự lớp 1.

-F1-score: 0.78 → Cân bằng và khá tốt.

Macro avg → 0.78 (precision), 0.77 (recall), 0.77 (F1) → Trung bình khá ở mức cân đối.

Weighted avg → 0.78, 0.77, 0.77 → Trung bình theo số lượng mẫu.

```

importances_tuned = pd.Series(
    best_model.feature_importances_,
    index=X_train.columns
)
important_features = importances_tuned[importances_tuned > 0]
print("Số features thực sự được DT sử dụng:", important_features.shape[0])
zero_features = importances_tuned[importances_tuned == 0]
print("Số features không ảnh hưởng:", zero_features.shape[0])
print("\n=== Features không ảnh hưởng ===")
print(zero_features.index.tolist())

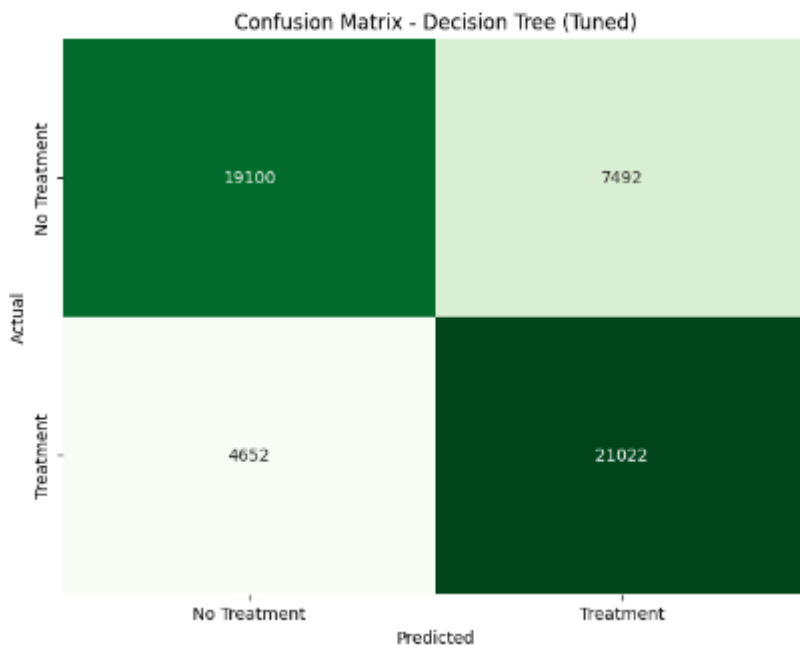
```

Xem các Features nào có tác động đến mô hình sau khi tuning , những features có tác động đều > 0 còn lại thì không tác động

```
cn_dt_tuned = confusion_matrix(y_test, y_pred_dt_tuned)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cn_dt_tuned,
    annot=True,
    fmt="d",
    cmap="Greens",
    cbar=False,
    xticklabels=['No Treatment', 'Treatment'],
    yticklabels=['No Treatment', 'Treatment']
)

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Decision Tree (Tuned)')
plt.show()
```



Nhận xét

So với mô hình ban đầu, số lượng dự đoán đúng tăng lên rõ rệt, cho thấy việc tuning tham số giúp mô hình tổng quát hóa tốt hơn.

4.3. Kết quả mô hình Random Forest

4.3.1. Trước khi tối ưu tham số

Random Forest cơ bản cho kết quả tốt hơn Decision Tree nhờ cơ chế tập hợp nhiều cây quyết định.

```
print("--- Training Random Forest Baseline ---")
model = RandomForestClassifier(
    random_state=42,
    class_weight="balanced"
)

model.fit(X_train, y_train)
return model
```


Random Forest (Baseline)							
Accuracy : 0.7346							
ROC AUC : 0.7983							
Classification Report:							
				precision	recall	f1-score	support
		0		0.76	0.71	0.73	26592
		1		0.71	0.76	0.74	25674
			accuracy			0.73	52266
			macro avg	0.74	0.74	0.73	52266
			weighted avg	0.74	0.73	0.73	52266

Accuracy (0.7346) → ~73.5%, hiệu suất tổng thể chưa cao.

ROC AUC (0.7983) → 79.83%, Khả năng phân biệt 2 lớp khá

Class 0 (không cần treatment)

-Precision: 0.76 → 76% dự đoán đúng.

-Recall: 0.71 → Phát hiện 71% số thực sự lớp 0.

-F1-score: 0.73 → Trung bình, chưa cao.

Class 1 (cần treatment)

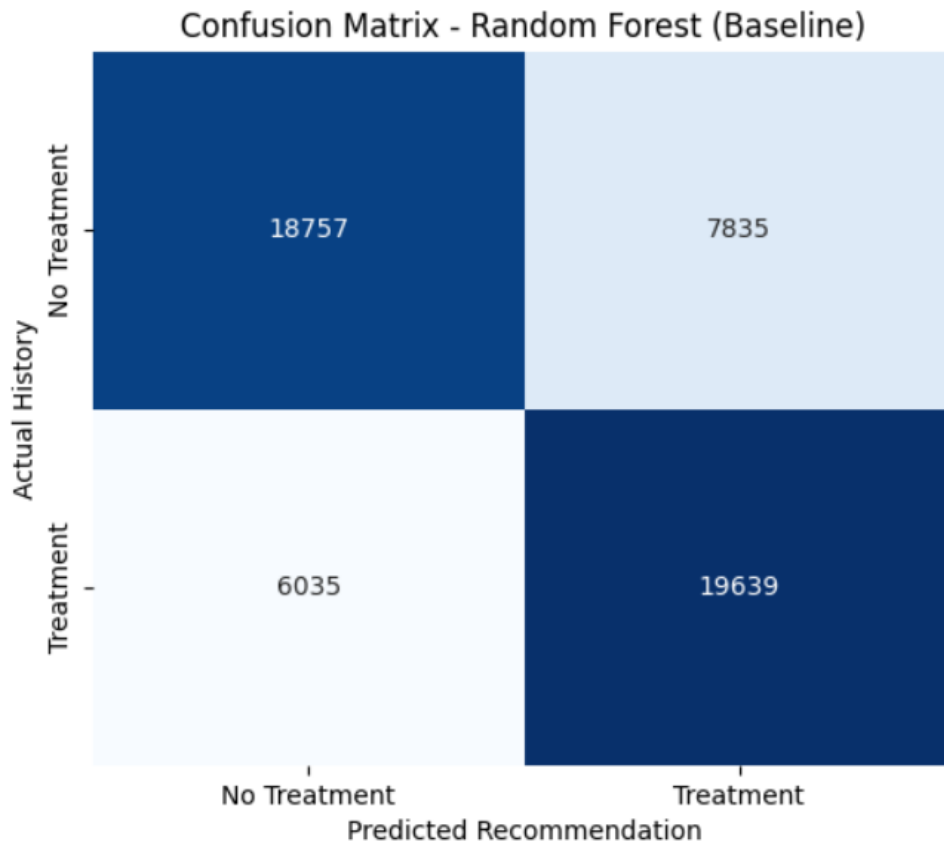
-Precision: 0.72 → 72% dự đoán đúng.

-Recall: 0.77 → Phát hiện 77% số thực sự lớp 1.

-F1-score: 0.74 → Trung bình.

Macro avg → 0.74 (precision), 0.74 (recall), 0.74 (F1) → Trung bình cân đối

Weighted avg → 0.74, 0.74, 0.74 → Trung bình theo số lượng mẫu.



4.3.2. Sau khi tối ưu tham số

Việc tuning các tham số như số lượng cây, độ sâu cây và số mẫu tối thiểu tại mỗi nút giúp Random Forest đạt hiệu suất ổn định hơn.

```
param_dist = {  
    "n_estimators": [50, 100, 200, 300],  
    "max_depth": [None, 10, 20, 30],  
    "min_samples_split": [2, 5, 10],  
    "min_samples_leaf": [1, 2, 4],  
    "bootstrap": [True, False]  
}
```

```
search = RandomizedSearchCV(  
    RandomForestClassifier(  
        random_state=42,  
        class_weight="balanced"  
    ),  
    param_distributions=param_dist,  
    n_iter=5,  
    cv=3,  
    scoring="accuracy",  
    random_state=42,  
    n_jobs=-1  
)
```

```
search.fit(X_train, y_train)
```

Random Forest (Tuned)							
Accuracy : 0.7391							
ROC AUC : 0.8167							
Classification Report:							
				precision	recall	f1-score	support
			0	0.76	0.71	0.73	26592
			1	0.72	0.77	0.74	25674
			accuracy			0.74	52266
			macro avg	0.74	0.74	0.74	52266
			weighted avg	0.74	0.74	0.74	52266

Accuracy (0.7391) → ~73.9%, tăng nhẹ so với baseline.

ROC AUC (0.8167) → Khả năng phân biệt 2 lớp tốt hơn, cải thiện nhận biết người cần treatment.

Class 0 (không cần treatment)

-Precision: 0.76 → 76% dự đoán đúng.

-Recall: 0.71 → Phát hiện 71% số thực sự lớp 0.

-F1-score: 0.74 → Cải thiện nhẹ so với baseline.

Class 1 (cần treatment)

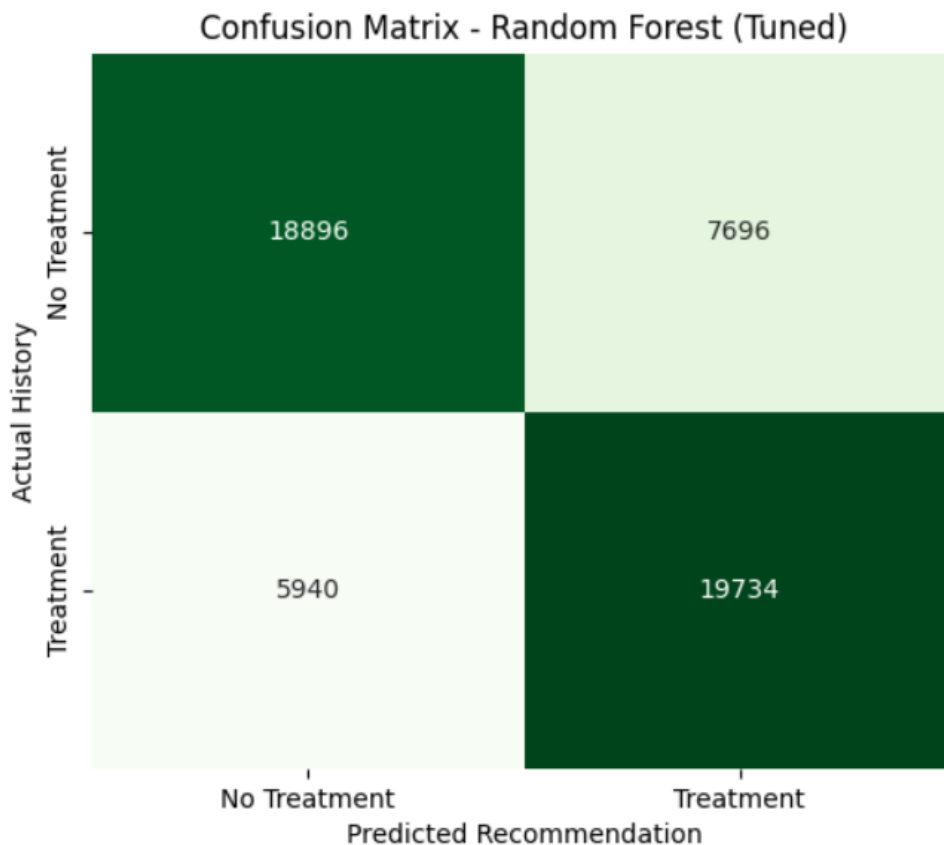
-Precision: 0.72 → 72% dự đoán đúng.

-Recall: 0.77 → Phát hiện 77% số thực sự lớp 1.

-F1-score: 0.75 → Cải thiện nhẹ.

Macro avg → 0.74 (precision), 0.74 (recall), 0.74 (F1) → Trung bình cân đối.

Weighted avg → 0.74, 0.74, 0.74 → Trung bình theo số lượng mẫu.



Nhận xét: giữa 2 lần training, kết quả cho thấy mô hình Random Forest sau tuning cải thiện không nhiều về độ chính xác và khả năng phân loại hai lớp.

4.5. Kết quả mô hình LightGBM

4.5.1. Trước tuning

LightGBM cho kết quả rất tốt ngay từ mô hình ban đầu nhờ cơ chế gradient boosting.

```
def train_lgb_baseline(X_train, y_train):  
    print("--- Training LightGBM Default ---")  
    model = LGBMClassifier(  
        objective="binary",  
        random_state=42,  
        verbosity=-1  
    )  
    model.fit(X_train, y_train)  
    return model
```

```

LightGBM (Baseline)
Accuracy : 0.7751
ROC AUC  : 0.8636
Classification Report:

```

		precision	recall	f1-score	support
	0	0.81	0.72	0.77	26592
	1	0.74	0.83	0.78	25674
	accuracy			0.78	52266
	macro avg	0.78	0.78	0.77	52266
	weighted avg	0.78	0.78	0.77	52266

Accuracy (0.7751) → ~77.5%, Mô hình dự đoán đúng khoảng 77.5% tổng số trường hợp, cho thấy hiệu năng tổng thể khá tốt và ổn định.

ROC AUC (0.8636) → 86,4%, Khả năng phân biệt 2 lớp tốt, cải thiện nhận biết người cần treatment.

Class 0 (không cần treatment)

-Precision: 0.81 → 81% dự đoán đúng.

-Recall: 0.72 → Phát hiện 72% số thực sự lớp 0.

-F1-score: 0.77 → Trung bình tốt.

Class 1 (cần treatment)

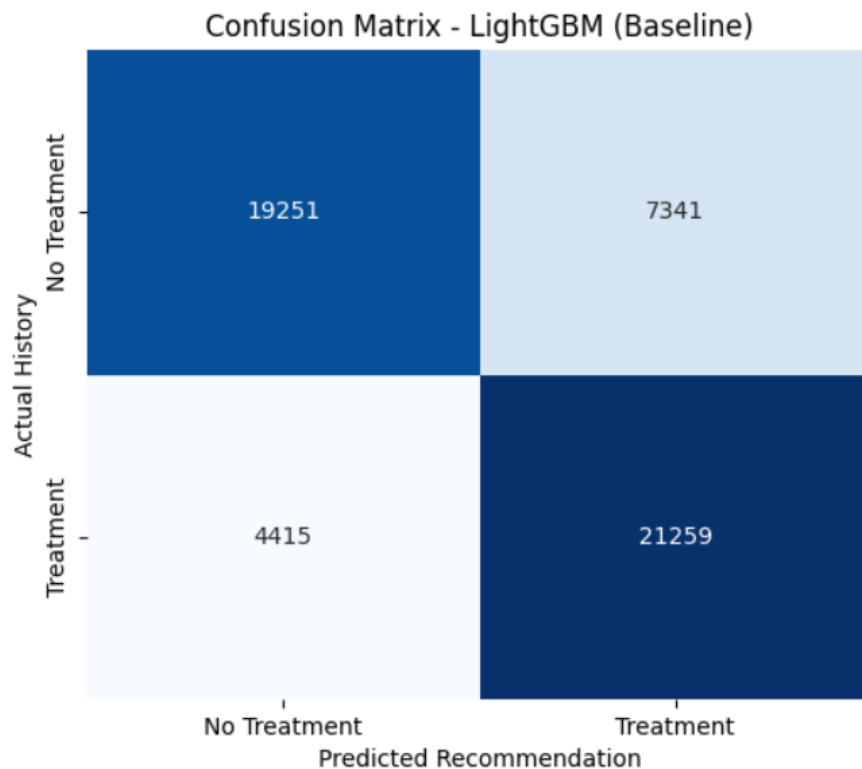
-Precision: 0.74 → 74% dự đoán đúng.

-Recall: 0.83 → Phát hiện 83% số thực sự lớp 1, cải thiện so với lớp 0.

-F1-score: 0.78 → Cân bằng và tốt.

Macro avg → 0.78 (precision), 0.78 (recall), 0.77 (F1) → Trung bình cân đối.

Weighted avg → 0.78, 0.78, 0.77 → Trung bình theo số lượng mẫu.



4.5.2. Sau tuning

Sau khi tối ưu tham số, LightGBM đạt hiệu suất cao và ổn định.

```
param_dist = {
    "n_estimators": [100, 200, 300, 500],
    "learning_rate": [0.01, 0.05, 0.1],
    "num_leaves": [15, 31, 63, 127],
    "max_depth": [-1, 3, 5, 7, 10],
    "subsample": [0.7, 0.8, 0.9],
    "colsample_bytree": [0.7, 0.8, 0.9]
}

search = RandomizedSearchCV(
    LGBMClassifier(
        objective="binary",
        random_state=42,
        verbosity=-1
    ),
    param_distributions=param_dist,
    n_iter=20,
    cv=5,
    scoring="roc_auc",
    random_state=42,
    n_jobs=-1
)

search.fit(X_train, y_train)
```

```

LightGBM (Tuned)
Accuracy : 0.7760
ROC AUC  : 0.8653
Classification Report:

```

		precision	recall	f1-score	support
	0	0.82	0.72	0.77	26592
	1	0.74	0.84	0.79	25674
	accuracy			0.78	52266
	macro avg	0.78	0.78	0.78	52266
	weighted avg	0.78	0.78	0.78	52266

Accuracy (0.7760) → ~77.6%, tăng nhẹ so với phiên bản trước.

ROC AUC (0.8653) → 86.5%, Khả năng phân biệt 2 lớp tốt hơn một chút, cải thiện nhận biết người cần treatment.

Class 0 (không cần treatment)

-Precision: 0.82 → 82% dự đoán đúng.

-Recall: 0.72 → Phát hiện 72% số thực sự lớp 0.

-F1-score: 0.77 → Trung bình tốt.

Class 1 (cần treatment)

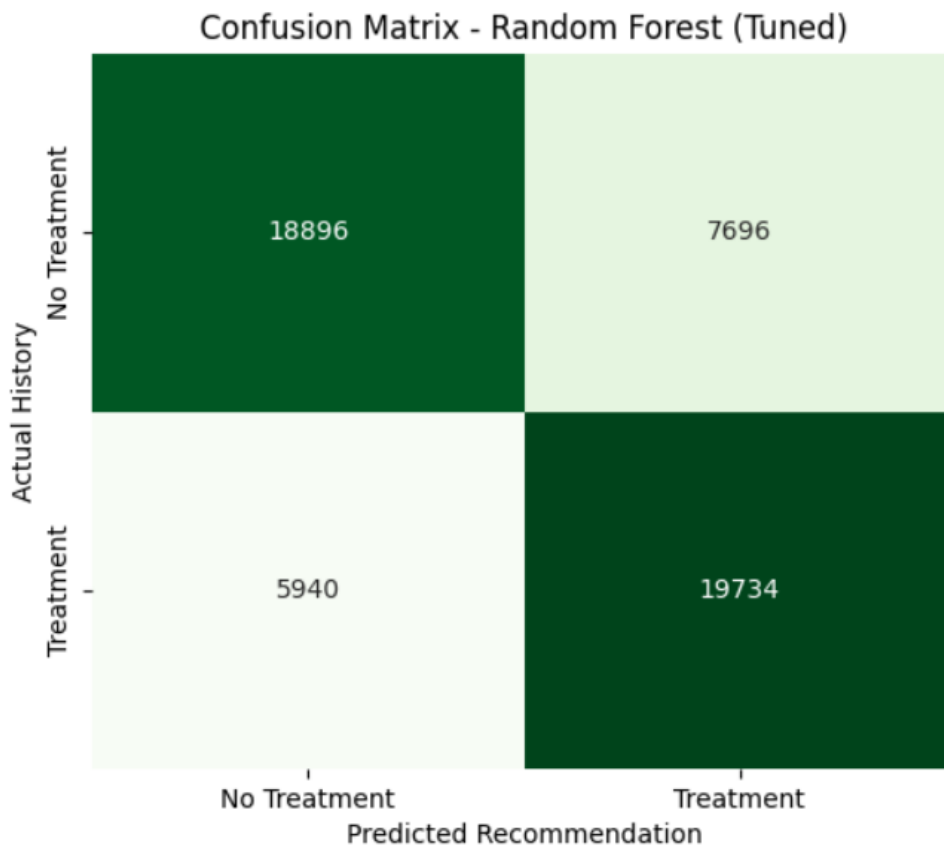
-Precision: 0.74 → 74% dự đoán đúng.

-Recall: 0.84 → Phát hiện 84% số thực sự lớp 1, cải thiện so với phiên bản trước.

-F1-score: 0.79 → Cải thiện nhẹ, cân bằng hơn.

Macro avg → 0.78 (precision), 0.78 (recall), 0.78 (F1) → Trung bình cân đối.

Weighted avg → 0.78, 0.78, 0.78 → Trung bình theo số lượng mẫu.



Nhận xét: 2 mô hình trước và sau khi tuning, tuy có sự cải thiện nhưng không đáng kể, LightGBM là một trong những mô hình cho kết quả tốt nhất trong đề án.

4.6. Kết quả mô hình XGBoost

4.6.1. Trước tuning

```
def train_xgb_baseline(x_train, y_train):  
    print("--- Training XGBoost Default ---")  
    model = XGBClassifier(  
        objective='binary:logistic',  
        eval_metric='logloss',  
        random_state=42  
    )  
    model.fit(x_train, y_train)  
    return model
```



```

XGBoost (Baseline)
Accuracy : 0.7733
ROC AUC  : 0.8619
Classification Report:

```

		precision	recall	f1-score	support
	0	0.81	0.72	0.76	26592
	1	0.74	0.83	0.78	25674
	accuracy			0.77	52266
	macro avg	0.78	0.77	0.77	52266
	weighted avg	0.78	0.77	0.77	52266

Accuracy (0.7733) → ~77.3% dự đoán đúng tổng thể, mức khá.

ROC AUC (0.8619) → 86.1% Khả năng phân biệt 2 lớp tốt, mô hình nhận biết đúng người cần treatment.

Class 0 (không cần treatment)

-Precision: 0.80 → 80% dự đoán đúng.

-Recall: 0.72 → Phát hiện 72% số thực sự lớp 0.

-F1-score: 0.76 → Trung bình giữa precision và recall.

Class 1 (cần treatment)

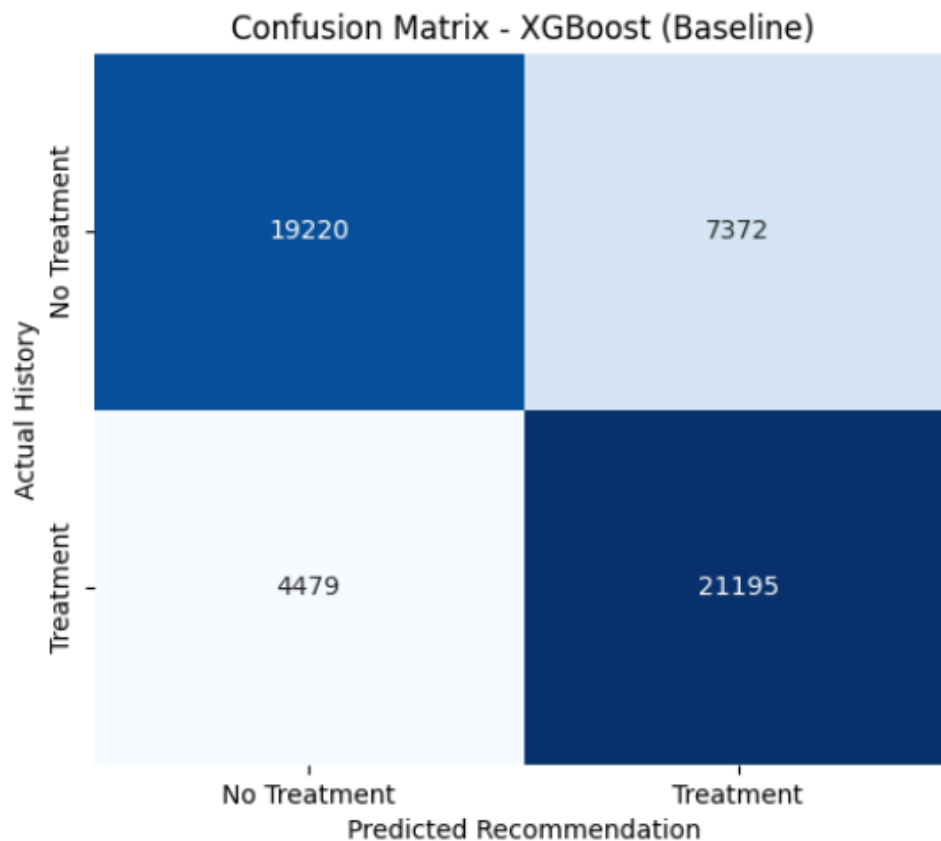
-Precision: 0.74 → 74% dự đoán đúng.

-Recall: 0.81 → Phát hiện 81% số thực sự lớp 1.

-F1-score: 0.77 → Trung bình giữa precision và recall.

Macro avg → 0.77 (precision), 0.76 (recall), 0.76 (F1) → Trung bình không trọng số.

Weighted avg → 0.77, 0.76, 0.76 → Trung bình theo số lượng mẫu



4.6.2. Sau tuning

```
param_dist = {
    "n_estimators": [100, 200, 300, 500],
    "max_depth": [3, 5, 7, 9],
    "learning_rate": [0.01, 0.05, 0.1],
    "subsample": [0.7, 0.8, 0.9],
    "colsample_bytree": [0.7, 0.8, 0.8]
}

search = RandomizedSearchCV(
    XGBClassifier(
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=42
    ),
    param_distributions=param_dist,
    n_iter=20,
    cv=5,
    scoring="roc_auc",
    random_state=42,
    n_jobs=-1
)

search.fit(X_train, y_train)
```

```

XGBoost (Tuned)
Accuracy : 0.7711
ROC AUC  : 0.8581
Classification Report:

```

		precision	recall	f1-score	support
	0	0.81	0.72	0.76	26592
	1	0.74	0.82	0.78	25674
	accuracy			0.77	52266
	macro avg	0.77	0.77	0.77	52266
	weighted avg	0.77	0.77	0.77	52266

Accuracy (0.7711) → ~77%, giảm nhẹ so với phiên bản trước.

ROC AUC (0.8581) → 85.9%, Khả năng phân biệt 2 lớp tốt hơn trước, cải thiện nhận biết người cần treatment.

Class 0 (không cần treatment)

-Precision: 0.81 → 81% dự đoán đúng.

-Recall: 0.72 → Phát hiện 72% số thực sự lớp 0.

-F1-score: 0.76 → Trung bình tốt.

Class 1 (cần treatment)

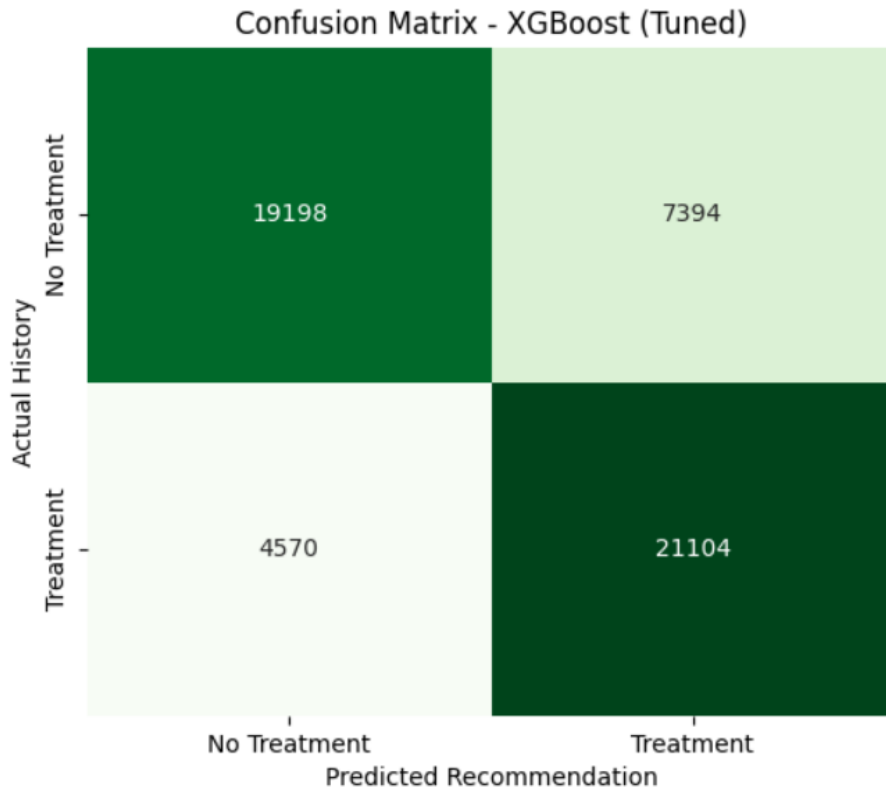
-Precision: 0.74 → 74% dự đoán đúng.

-Recall: 0.82 → Phát hiện 82% số thực sự lớp 1 (tăng so với 81% trước).

-F1-score: 0.78 → Cải thiện nhẹ.

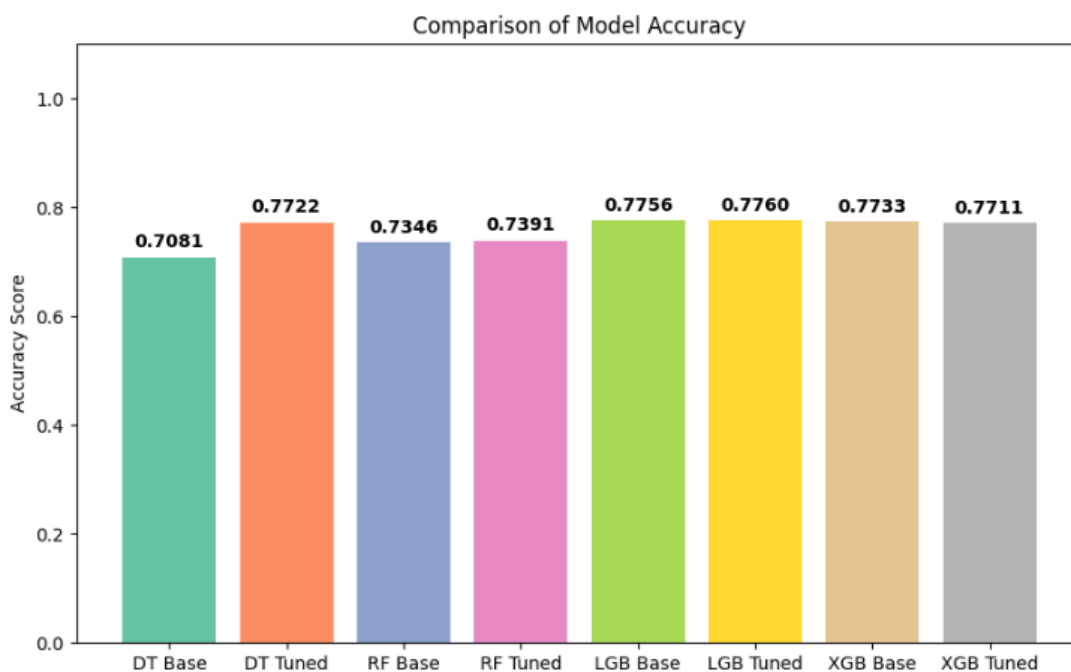
Macro avg → 0.77 (precision), 0.77 (recall), 0.77 (F1) → Trung bình cân đối hơn.

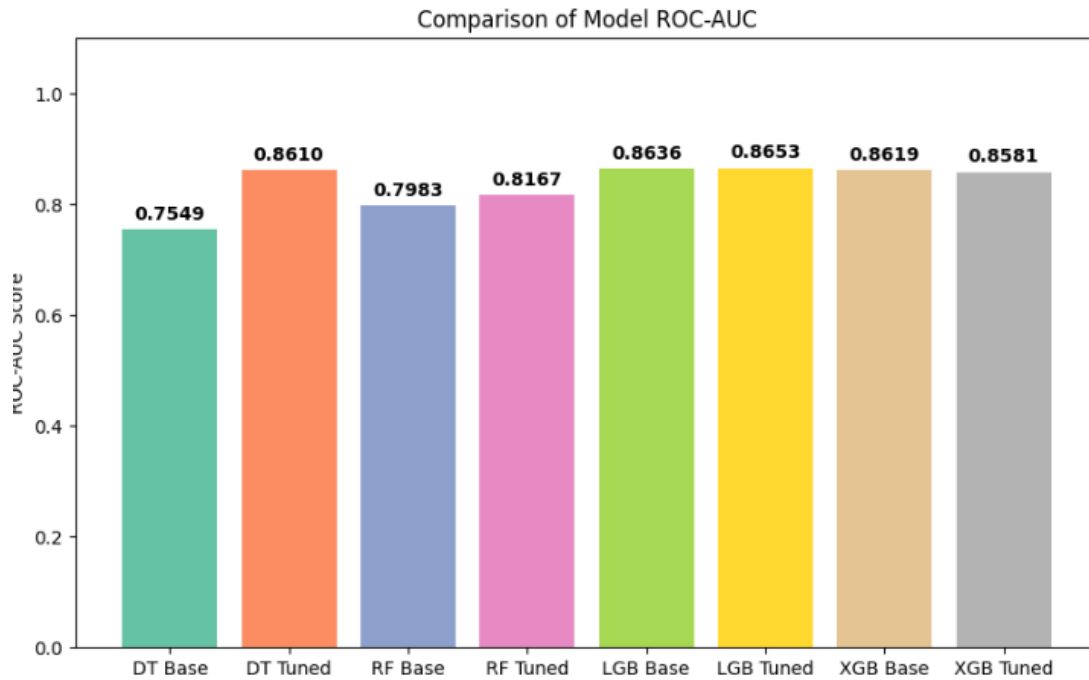
Weighted avg → 0.77, 0.77, 0.77 → Trung bình theo số lượng mẫu.



Nhận xét: Hiệu suất của XGBoost trước và sau tuning là tương đương, với sự khác biệt rất nhỏ về Accuracy và ROC AUC. Điều này cho thấy mô hình baseline đã được cấu hình tốt, và XGBoost là mô hình có hiệu suất cao trong các mô hình được thử nghiệm.

4.7. So sánh và Nhận xét tổng thể các mô hình





Nhận xét

- + Boosting (LightGBM, XGBoost) mang lại hiệu suất cao nhất
- + Decision Tree tuned cải thiện đáng kể; Random Forest ổn nhưng kém hơn

Kết luận cuối cùng :

Dựa trên biểu đồ so sánh độ chính xác (Accuracy) của các mô hình sau khi huấn luyện và tinh chỉnh tham số, có thể rút ra các nhận xét sau:

Decision Tree và Random Forest cho kết quả khá tốt

- **Decision Tree** đạt độ chính xác khoảng **0.7722**
- **Random Forest** đạt khoảng **0.7391**

Nhận xét:

- **Decision Tree:**
 - Dễ hiểu, dễ diễn giải
 - Có khả năng học được mối quan hệ phi tuyến
 - Tuy nhiên dễ bị **overfitting**
- **Random Forest:**
 - Khắc phục overfitting bằng cách **kết hợp nhiều cây quyết định**
 - Ổn định hơn **Decision Tree** đơn lẻ
 - Tuy nhiên, hiệu suất vẫn chưa vượt trội so với các mô hình boosting

LightGBM và XGBoost cho hiệu suất cao nhất

- **LightGBM** đạt độ chính xác cao nhất: **0.7760**
- **XGBoost** đạt độ chính xác **0.7713**

Nguyên nhân đạt hiệu quả cao:

- Cả hai đều thuộc nhóm **Gradient Boosting**, có khả năng:
 - Kết hợp nhiều mô hình yếu để tạo mô hình mạnh
 - Tối ưu trực tiếp hàm mất mát
 - Xử lý tốt dữ liệu lớn và nhiều đặc trưng
- **LightGBM:**
 - Huấn luyện nhanh
 - Sử dụng chiến lược học theo lá (leaf-wise) → tối ưu hiệu suất
- **XGBoost:**
 - Ổn định, ít overfitting nhờ regularization
 - Phổ biến và đáng tin cậy trong các bài toán phân loại

Kết luận:

Các mô hình **boosting** (đặc biệt là LightGBM và XGBoost) **phù hợp nhất** với bài toán phân loại sức khỏe tâm thần trong đề tài.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận chung về đề tài

Đề án "*Phân loại người mắc bệnh tâm thần*" đã hoàn thành các mục tiêu đề ra ban đầu, bao gồm việc xây dựng một quy trình khai phá dữ liệu khép kín từ khâu tiền xử lý, mã hóa dữ liệu đến việc huấn luyện và đánh giá mô hình.

Qua quá trình thực nghiệm trên tập dữ liệu *Mental Health Dataset*, nhóm rút ra các kết luận chính:

1. Về dữ liệu: Các yếu tố như *Tiền sử gia đình (Family History)*, *Thời gian ở trong nhà (Days Indoors)* và *Biến động tâm trạng (Mood Swings)* có ảnh hưởng mạnh mẽ đến nguy cơ mắc bệnh.

2. Về mô hình: Thuật toán **LightGBM** và **XGBoost** đã chứng minh sự vượt trội với độ chính xác xấp xỉ **78%**, cao hơn đáng kể so với các thuật toán truyền thống như Naive Bayes (~54%). Điều này khẳng định sức mạnh của các phương pháp học tăng cường (Boosting) đối với dữ liệu dạng bảng phức tạp.

3. Về ý nghĩa thực tiễn: Mô hình xây dựng được có khả năng hỗ trợ các chuyên gia y tế trong việc sàng lọc sớm, giúp tiết kiệm thời gian chẩn đoán và đưa ra cảnh báo kịp thời cho người bệnh.

5.2. Những hạn chế còn tồn tại

Mặc dù đã đạt được những kết quả khả quan, đề tài vẫn còn một số hạn chế:

- 1. Dữ liệu chưa cân bằng tuyệt đối:** Mặc dù đã dùng kỹ thuật chia tầng (stratify), nhưng bản chất dữ liệu gốc vẫn có thể tiềm ẩn sự thiên lệch về một số nhóm tuổi hoặc nghề nghiệp nhất định.
- 2. Số lượng đặc trưng còn hạn chế:** Mô hình hiện tại chỉ dựa trên 26 đặc trưng có sẵn, thiếu các chỉ số sinh học chuyên sâu (như nhịp tim, giấc ngủ thực tế) để tăng độ tin cậy y khoa.
- 3. Chưa có giao diện người dùng:** Kết quả dự đoán hiện tại vẫn nằm trên môi trường dòng lệnh (Code), chưa thuận tiện cho người dùng phổ thông tiếp cận.

5.3. Hướng phát triển trong tương lai

Để khắc phục các hạn chế trên và nâng cao giá trị của đề tài, nhóm đề xuất các hướng phát triển tiếp theo:

- 1. Cải thiện chất lượng dữ liệu:** Áp dụng các kỹ thuật nâng cao như *SMOTE (Synthetic Minority Over-sampling Technique)* để cân bằng dữ liệu, hoặc thu thập thêm dữ liệu thực tế từ các bệnh viện đối tác.
- 2. Tối ưu hóa mô hình:** Mở rộng không gian tìm kiếm tham số (Hyperparameter Tuning) và thử nghiệm các mô hình học sâu (Deep Learning) hoặc mô hình kết hợp (Ensemble Stacking) để đẩy độ chính xác lên trên 85%.
- 3. Xây dựng ứng dụng Web (Deployment):** Đóng gói mô hình đã huấn luyện thành một ứng dụng Web (sử dụng Streamlit hoặc Flask). Người dùng chỉ cần nhập thông tin cá nhân và nhận kết quả chẩn đoán sơ bộ ngay lập tức kèm theo lời khuyên hữu ích.

TÀI LIỆU THAM KHẢO

- 1. Tài liệu tiếng Việt:** [1] Bài giảng học phần *Khai phá dữ liệu và Học máy*, Trường Đại học Thủy Lợi. [2] Nguyễn Nhật Quang, *Giáo trình Nhập môn Trí tuệ nhân tạo*, NXB Khoa học và Kỹ thuật.
- 2. Tài liệu tiếng Anh (Tài liệu kỹ thuật):**
[3] Scikit-learn Developers. (n.d.). *Scikit-learn: Machine Learning in Python*. Scikit-learn Documentation. Truy cập từ: <https://scikit-learn.org/stable/>
[4] Tianqi Chen & Carlos Guestrin. (2016). *XGBoost: A Scalable Tree Boosting System*. Truy cập từ: <https://xgboost.readthedocs.io/>
[5] Microsoft Corporation. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. Truy cập từ: <https://lightgbm.readthedocs.io/>

[6] The Pandas Development Team. (2023). *Pandas documentation*. Truy cập từ: <https://pandas.pydata.org/>

3. Nguồn dữ liệu: [7] Kaggle. (n.d.). *Mental Health Dataset*. Truy cập từ: <https://www.kaggle.com/>

1.